SYSTEM REQUIREMENT SPECIFICATION (SRS)

File Organization Automation

Ye Zeng TAN 104398518
Angela 104397586
Jia You LIM 104401160
Qiung Hung CHONG 104397861
Wen Xiun THEN 104396567

Table of Contents

System Requirements Specification (SRS) 1
Introduction 1
Background 1
Problem Statement 1
Stakeholders 2
Deliverables 2
Objectives 2
Scope 2
System Description 2
Planning 4
Methodology 4
Milestones and Timeline 5
Work Breakdown 6
Roles 7
Communication 8
Risk Management 8
Requirements 11
User Stories 11
Functional Requirements 12
Summary of use cases 18
Non-functional Requirements 19
Constraints 19
Conclusion 20
References 21

Figure List

Figure 1: The user flow diagram illustrates the process of managing files through OneDrive. The user scans a folder and views the results on the dashboard. If any files are missing or have naming issues, the user chooses to fix them by selecting default or custom rules. Once all problems are resolved, the process ends. 4
Figure 2: Project Gantt Chart Showing Timeline and Task Distribution from Week 2 to Week 12. 6
Figure 3: Work Breakdown Structure (WBS) that hierarchically decompose the entire project. 7
Figure 4: This poster presents a detailed SWOT analysis of the Template-Based Cloud File Organization and Compliance System project. It visually summarizes the project's Strengths, Weaknesses, Opportunities, and Threats to provide a clear overview of internal and external factors affecting its development and implementation. 11
Figure 5: Login Page. 12
Figure 6: Dashboard View. 13
Figure 7: Folder Selection Page. 13
Figure 8: Template Selection Page. 14
Figure 9: Scanning Pop Up. 15
Figure 10: Scan Result Page. 15
Figure 11: Rules Page. 16
Figure 12: Final Result Page. 17
Figure 13: Report Page. 17
Figure 14: Log Out Pop Up. 17

Table List

Table 1： The table shows the project timeline from week 2 -12. 5
Table 2: Assignment of team roles and responsibilities for system development and documentation activities. 8
Table 3: Identification of potential project risks, their impact levels, and corresponding mitigation strategies. 9

System Requirements Specification (SRS)

Introduction

The increasing reliance on cloud storage platforms such as Microsoft OneDrive has significantly transformed how users manage digital files for academic, professional, and personal purposes. However, as users continuously upload files over time, these storage systems often become disorganized due to inconsistent folder structures, unclear naming conventions, and missing documents. This disorganization reduces efficiency, particularly when users need to locate files or verify whether required documents exist. Therefore, this project proposes a Template-Based Cloud File Organization and Compliance System that introduces automation and standardization into cloud file management. The system aims to help users maintain a structured and compliant file system with minimal manual effort, directly addressing the business need for efficient file organization described in the project proposal.

Background

In modern digital environments, users rely heavily on cloud storage platforms such as Microsoft OneDrive to store documents for academic, professional, and personal purposes. However, real-world feedback from client consultations reveals that these storage systems often become increasingly difficult to manage over time, which client specifically noting that his files become messy and every folder becomes disorganized due to inconsistent folder hierarchies, missing files, and non-standard naming conventions. As a result, this lack of organization leads to inefficiencies, including increased time spent searching for files and difficulty verifying document completeness. The proposed system is designed to address these issues by introducing automation into file organization processes, ensuring consistency, and improving accessibility. By integrating rule-based file management and predefined templates, the system aims to provide a structured and efficient approach to cloud file management.

Problem Statement

The primary problem addressed by this system is the inefficiency and inconsistency in managing files within cloud storage environments. Without a standardized structure, users often encounter challenges such as misplaced documents, incomplete file sets, and difficulty tracking required files. These issues can result in wasted time, reduced productivity, and potential non-compliance with organizational requirements. If left unaddressed, these problems may escalate as the volume of stored files increases, further complicating file management and reducing operational efficiency. Therefore, there is a clear need for an automated system that can enforce structure, detect missing files, and streamline file organization processes.

Stakeholders

The primary stakeholders of this system include the business client, system users, development team, IT application support team, and the lecturer (supervisor). The business client is concerned with the overall value of the system and its potential for real-world application. System users, such as employees or internal users, prioritize ease of use, efficiency, and accuracy in file organization. The development team, consisting of students, is responsible for implementing the system according to the specified requirements while ensuring maintainability and scalability. The IT application support team plays a critical role in ensuring that the system complies with development standards and evaluating its suitability for potential production deployment. The lecturer, acting as the project supervisor, provides guidance, ensures academic quality, and monitors project progress. Although each stakeholder group has distinct responsibilities and interests, they all share a common objective of improving file management efficiency and ensuring system reliability.

Deliverables

Objectives

The objective of this system is to develop a web-based application that automates the organization and validation of files stored in cloud environments. The system aims to provide measurable improvements such as reducing the time required to locate files, increasing the accuracy of file organization, and ensuring compliance with predefined folder structures. Additionally, the system seeks to provide a user-friendly interface that enables users to easily manage their files, monitor organization status, and generate compliance reports. Another key objective is to ensure that the system adheres to IT application standards, making it maintainable and suitable for potential future deployment.

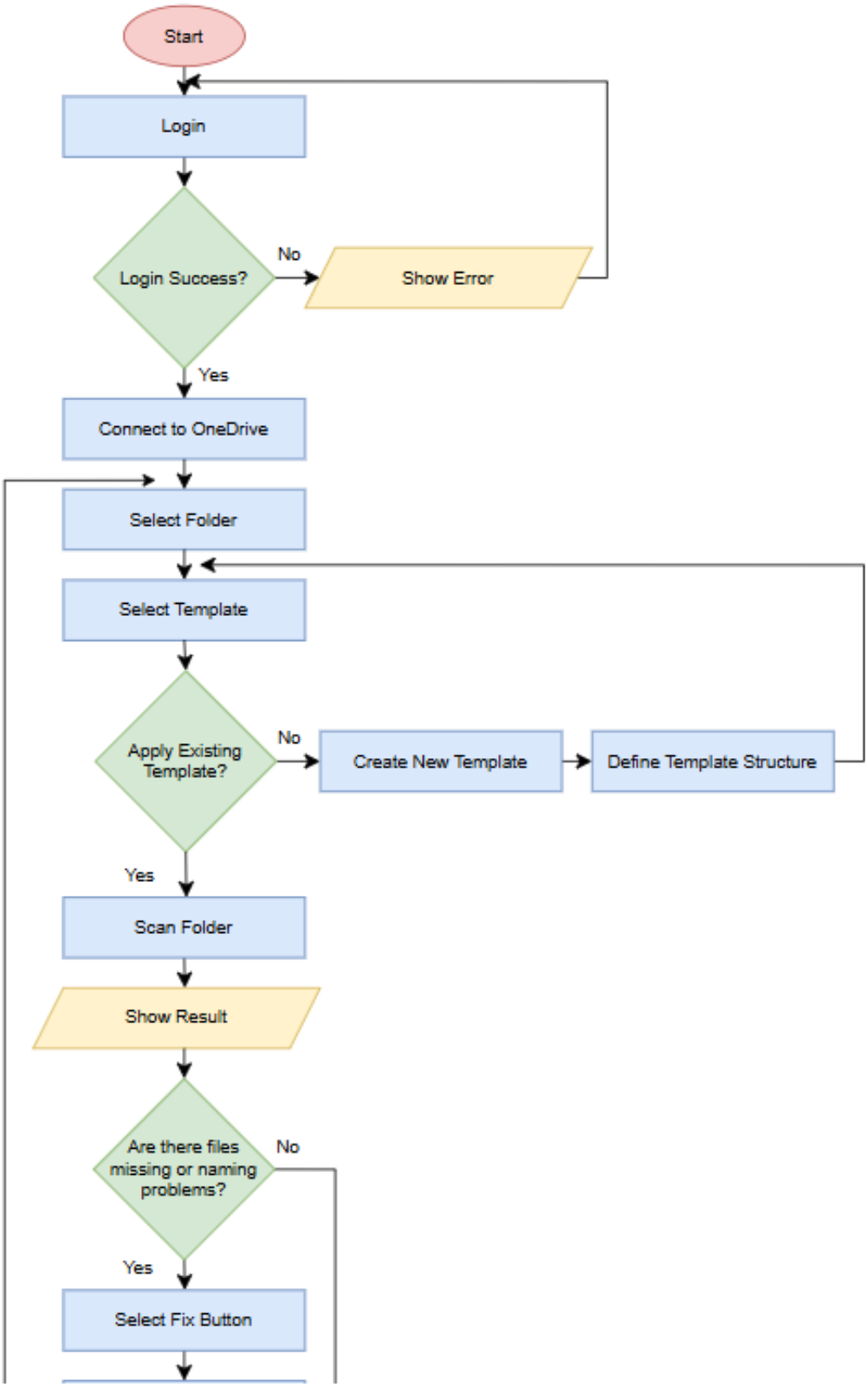
Scope

The scope of this project includes functionalities that allow users to connect to their OneDrive accounts, select folders, apply predefined templates, and scan file structures. The system will detect missing or incorrectly named files and perform automated actions such as renaming, moving, or copying files based on user-defined rules. It will also provide a dashboard to visualize file organization status and generate compliance reports.

System Description

The system is a web-based application developed using the JavaScript/Typescript language within the Next.js framework, which integrates both frontend and backend functionalities. The frontend provides a user interface that allows users to interact with the system, while the backend handles processing logic, including file analysis and rule execution. The system connects to Microsoft OneDrive using the Microsoft Graph API, enabling access to user files

and folders. A rule-based engine processes file operations such as renaming, moving, and copying, while a lightweight (JSON database) database stores templates, rules, and logs. The typical user flow involves logging in, connecting to OneDrive, selecting a folder, applying a template, scanning the structure, and viewing the results through a dashboard.



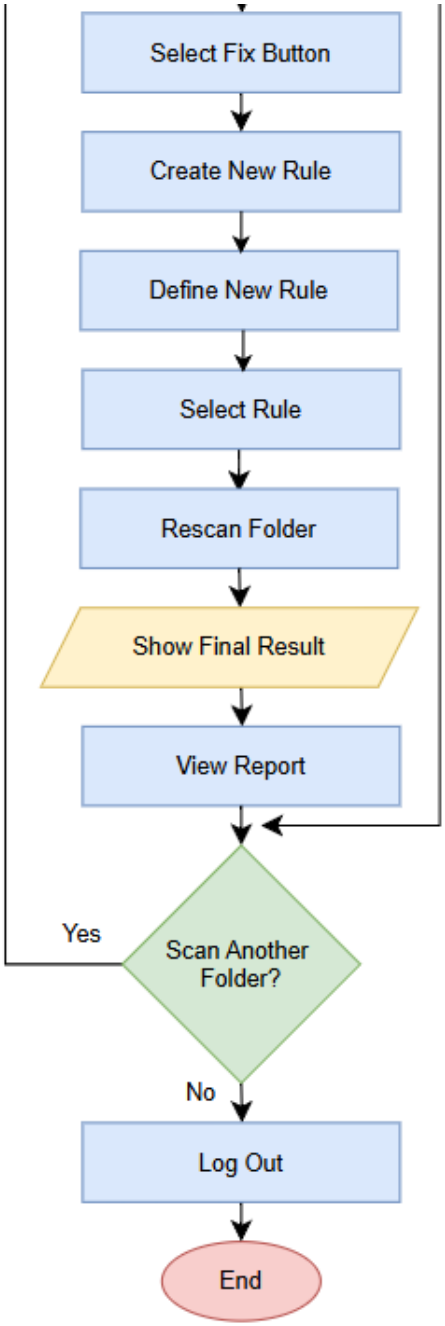


Figure 1: The user flow diagram illustrates the process of managing files through OneDrive. The user scans a folder and views the results on the dashboard. If any files are missing or have naming issues, the user chooses to fix them by selecting default or custom rules. Once all problems are resolved, the process ends.

Planning

Methodology

The project will adopt an Agile-based development methodology to ensure flexibility, iterative progress, and continuous collaboration throughout the development lifecycle (GeeksforGeeks, 2025). This approach is suitable because the project consists of multiple phases, including

requirement analysis, design, development, testing, and documentation, all of which may require refinement based on feedback from the supervisor and team members. Agile enables the team to divide the project into manageable iterations. Each iteration will focus on completing a core functional component, starting with the Microsoft Graph API integration and moving into the rule-based engine and compliance dashboard. It supports continuous feedback, allowing for early identification of issues and continuous delivery of functional components (GeeksforGeeks, 2025). Regular review sessions and collaborative discussions will be conducted to ensure that all team members remain aligned with the project objectives, technical requirements, and IT application compliance standards.

Milestones and Timeline

The project will be structured into several key phases, including requirement analysis, system design, development, testing, and final delivery, each with clearly defined objectives and deliverables. During the requirement analysis phase, the team will gather and validate system requirements, while the design phase focuses on producing system architecture diagrams and detailed technical specifications. In the development phase, core functionalities such as API integration and rule-based processing will be implemented, followed by rigorous testing to validate system performance and identify defects. A formal code review session with the IT application support team will ensure compliance with development standards.

To guide these phases effectively, the project timeline follows the SMART framework, which emphasizes setting goals that are Specific, Measurable, Achievable, Relevant, and Time-bound (Martins, 2026). Specific goals are set for each phase, giving the team a clear understanding of the expected outcomes. Measurable progress is maintained through weekly tracking, allowing milestones to be monitored and adjustments made as needed. Achievability is ensured by assigning tasks according to each team member's expertise and capacity, keeping the workload realistic and manageable. Relevance is maintained by aligning all activities with the project objectives and the intended system functionality, ensuring that effort is focused where it matters most. Finally, the timeline is strictly time-bound, running from Week 2 to Week 12, which guarantees that all deliverables will be completed within the academic deadline. This SMART-based structure not only clarifies responsibilities but also provides a practical roadmap, enabling the team to execute the project efficiently while staying aligned with its goals.

Phase	Weeks	Description
SRS Development	Week 2–4	Complete requirements and documentation
Design Report	Week 4–7	Produce system diagrams and architecture
Development	Week 4–10	Implement system features
Testing	Week 7–10	Validate functionality and fix defects
Final Deliverables	Week 10–12	Submit final report, system, poster

Table 1 : The table shows the project timeline from week 2 -12.

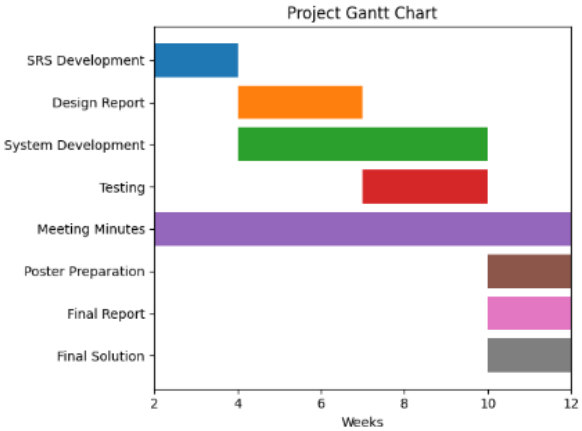


Figure 2: Project Gantt Chart Showing Timeline and Task Distribution from Week 2 to Week 12.

Work Breakdown

The work breakdown structure divides the project into major components, including SRS documentation, system design, system development, testing, and final deliverables. The SRS phase focuses on requirement gathering and documentation to ensure that system objectives, scope, and constraints are clearly defined. The design phase includes the development of system architecture diagrams, user flow diagrams, data flow diagrams, sequence diagrams, entity-relationship diagrams, and API design specifications.

The development phase involves implementing the frontend using Next.js, integrating APIs to communicate with Microsoft OneDrive, and designing the user interface. Testing activities focus on validating system functionality, identifying defects, and ensuring compliance with defined requirements. The final deliverables include the test report, system implementation, poster, and final documentation.

The project is organized according to the academic timeline from Week 2 to Week 12. During Weeks 2 to 4, the team will focus on developing the SRS, with Ye Zeng responsible for the introduction, deliverables and planning sections, while Wen Xiun and Jia You contribute to interface design, functional requirements, and system architecture components. From Weeks 4 to 7, all team members will collaboratively produce the design report, ensuring completeness before submission.

Between Weeks 4 and 10, system development will proceed alongside design activities. Jia You will lead the implementation using Next.js, Angela will focus on API integration, and Qiung Hung, together with Angela, will handle UI/UX design. Testing activities will take place after the design phase, with Ye Zeng and Qiung Hung responsible for validating system functionality and preparing the test report by Week 10.

Meeting documentation will be maintained continuously from Week 2 to Week 12 by Wen Xiun and Angela, while Ye Zeng will review and refine the documentation. In Week 12, final deliverables will be completed, including the project poster by Wen Xiun, the final report led by Ye Zeng, and the system implementation finalized by Jia You. This structured work breakdown ensures clear task allocation, accountability, and efficient collaboration.

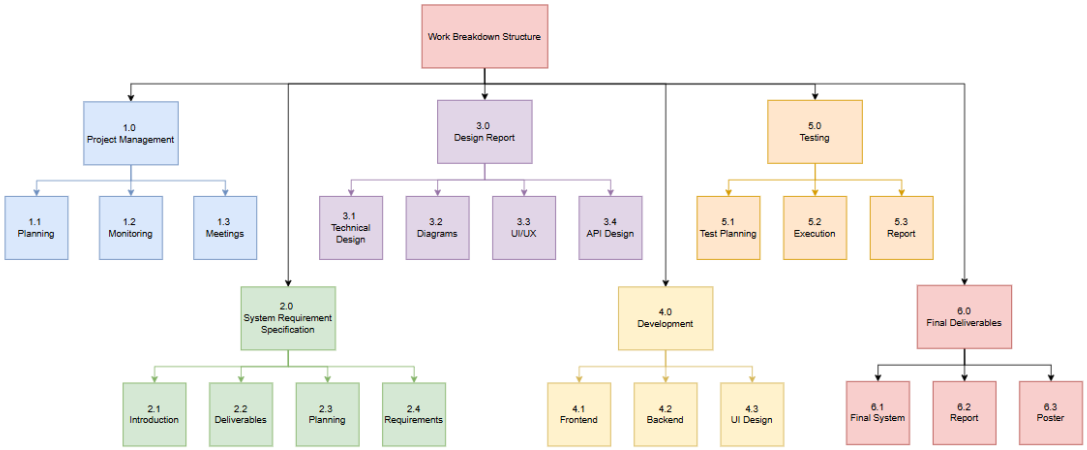


Figure 3: Work Breakdown Structure (WBS) that hierarchically decompose the entire project.

Roles

Each team member has been assigned specific roles to ensure effective collaboration and successful project completion. Ye Zeng, as the Team Leader, oversees overall project coordination and is responsible for key documentation tasks, including the SRS Introduction, Deliverables and Planning, reviewing meeting minutes, conducting system testing, and leading the final report preparation. Jia You takes on the role of Backend Developer, focusing on system development with Next.js and contributing to both functional and non-functional requirements in the SRS. Wen Xiun, the Frontend Developer, oversees interface-related components, including designing the wireframe, creating architecture diagrams, preparing the project poster, and documenting meeting minutes. Angela serves as API Developer, handling backend integration while also supporting UI design and maintaining accurate meeting documentation. Qiung Hung acts as Tester and UI/UX Specialist, ensuring the user interface meets design standards and participating in system testing to identify defects. This well-defined allocation of roles ensures that all aspects of the project are managed efficiently, promotes accountability, and strengthens teamwork throughout the development process.

IT Personnel	Roles	Responsibility
Ye Zeng TAN	Team Leader	• Oversees project coordination • Manage SRS documentation Deliverables • Planning • Review meeting minutes Supervising system testing • Lead the final report
Jia You LIM	Backend Developer	• Develop system using Next.js • Define functional and non-functional requirements in SRS

Wen Xiun THEN	Frontend Developer	• Design wireframe • Create architecture diagrams • Manage project posters • Document meeting minutes
Angela	API Developer	• Handle backend integration (Microsoft Graph API) Support UI design • Maintain accuracy of meeting documentation
Qiung Hung CHONG	• UI/UX Specialist • Tester	• Ensure interface meets design standards Conduct system testing to identify defects

Table 2: Assignment of team roles and responsibilities for system development and documentation activities.

Communication

Effective communication is maintained through a combination of synchronous meetings and asynchronous digital platforms to ensure transparency and accountability. Formal weekly meetings are conducted either via Microsoft Teams or physically, where progress is reviewed against the project timeline. Meeting minutes are consistently recorded by Wen Xiun and Angela using shared cloud documentation on Google docs to track decisions and action items. Ye Zeng reviews and refines these records to ensure clarity and alignment with project goals.

To maintain continuous progress tracking, the team utilizes Trello as Task Management platform. This allows all members to visualize the status of their assigned tasks such as Next.js frontend development or API integration and promptly communicate any technical challenges. If a team member becomes unresponsive or fails to meet deadlines, follow-up actions are initiated through direct communication. Persistent issues are escalated to the unit convenor for resolution. This structured approach, supported by industry-standard tools, ensures efficient collaboration throughout the project lifecycle.

Risk Management

Category	Potential Risk	Risk Level	Mitigation Strategy
Project Management	**Tight Project Timeline:** Completing design, development, and testing within a restricted period.	High	Follow Agile methodology to prioritize "Must-Have" features, ensuring a functional core system is delivered first.
Technical	**Microsoft Graph API Complexity:** Limited experience with authentication and file retrieval.	High	Conduct early "Technical Spikes" to master OAuth 2.0 and basic file listing before building complex rules.

	Next.js Learning Curve: Coordinating complex frontend and backend logic in a full-stack framework	Medium	Adhere to official Next.js documentation and standard project structures to ensure the code remains maintainable.
Security	**Data Privacy & Security:** Risks involving user data handling and cloud access	Medium	Strictly align with IT compliance standards by using official Microsoft login flows; no user passwords will be stored.
Quality	**Testing & Debugging Challenges:** Complexity in validating API-dependent functionality.	Medium	Perform continuous unit testing during development and use "Mock Data" to verify logic before live API calls.
Compliance	**IT Standard Non-Compliance:** Failure to meet strict internal requirements affecting evaluation	Medium	Conduct regular reviews with the supervisor and cross-check development against the IT Support Compliance list.
Resource	**Small Team Workload:** High individual pressure and potential bottlenecks due to team size.	Medium	Use Trello for transparent task tracking and maintain a clear WBS to ensure work is evenly distributed.
Operational	**Internet/Cloud Dependency:** System vulnerability to connectivity disruptions or service downtime.	Low	Implement robust error handling and status alerts to notify users of drops; use local logging for audit trails.

Table 3: Identification of potential project risks, their impact levels, and corresponding mitigation strategies.

Strengths:

The strengths of the Template-Based Cloud File Organization and Compliance System project lie in its clear and well-defined scope, as well as the structured allocation of team roles. Each team member has specific responsibilities, which ensures accountability and efficient progress throughout the project lifecycle. The system also leverages standard, widely recognized technologies such as Next.js for full-stack development and the Microsoft Graph API for OneDrive integration, ensuring that the architecture is scalable, maintainable, and consistent with modern software development practices. Additionally, the project addresses a real-world problem by providing automated file organization, template compliance verification, and dashboard reporting, making the solution practical and relevant to potential users. Alignment

with internal IT compliance standards further strengthens the project by ensuring secure and maintainable code.

Weaknesses:

Despite these advantages, the project faces several weaknesses that may impact its progress. Limited experience with the Microsoft Graph API poses a technical challenge, as seamless integration is crucial for OneDrive operations such as authentication, file retrieval, and rule-based file management. Such issues fall under technical risks, which involve design, implementation, testing, and integration challenges in software projects (GeeksforGeeks, 2025). Additionally, the project timeline is tight, requiring the team to master the Next.js learning curve and coordinate complex frontend and backend logic within a restricted period. The small team size further increases individual workload, placing pressure on members to deliver high-quality, debugged API functionality under strict deadlines.

Opportunities:

The project also presents significant opportunities for both team development and project impact. Team members will gain hands-on experience with modern frameworks, cloud integration, and rule-based automation systems, enhancing technical skills that are highly valued in industry. Working on a system with real-world applications provides practical experience in enterprise-like scenarios, such as designing dashboards, implementing compliance checks, and handling user authentication securely. Exposure to internal IT compliance standards and organizational practices offers further learning opportunities, prepares the team for future professional settings. Collaborating within a structured project environment also helps strengthen teamwork, communication, and project management capabilities.

Threats:

However, the project is exposed to several threats that could hinder its success. The complexity of integrating the Microsoft Graph API may introduce unforeseen technical difficulties, while handling user data raises vital security and data privacy concerns. The system ‘s dependency on stable internet and cloud service availability makes it vulnerable to connectivity disruptions. Additionally, compliance with IT standards imposes strict requirements, and failure to meet these standards could compromise both system security and the project’s evaluation. Awareness of these threats and implementing proactive mitigation strategies such as early API testing, secure authentication, error handling, and adherence to project scope will be essential to ensure successful project completion. Continuous monitoring of these risks is necessary to ensure that mitigation strategies remain effective throughout the project lifecycle (GeeksforGeeks, 2025).

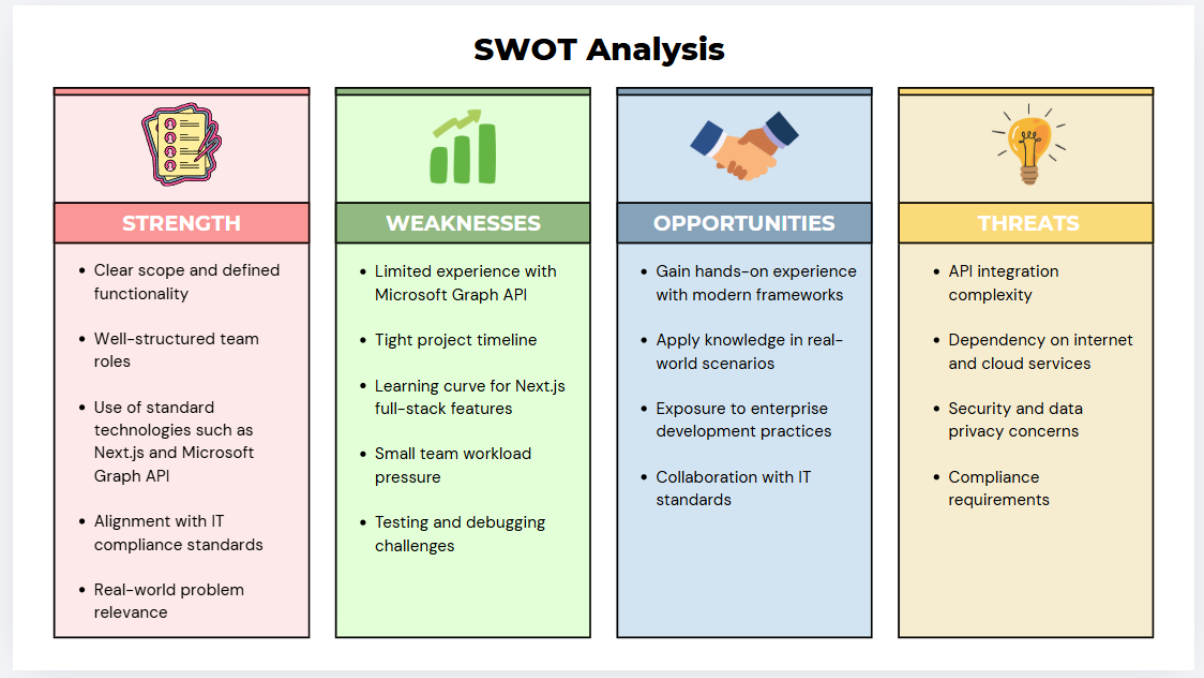


Figure 4: This poster presents a detailed SWOT analysis of the Template-Based Cloud File Organization and Compliance System project. It visually summarizes the project's Strengths, Weaknesses, Opportunities, and Threats to provide a clear overview of internal and external factors affecting its development and implementation.

Requirements

User Stories

The system enables users to securely access functionalities by signing in using a Microsoft account. Users can initiate a file scanning process to organize and validate files stored in selected folders. The system allows users to specify a folder as input so that only relevant files are processed. Users are also able to select or manage templates to apply predefined file recognition rules, ensuring consistent and efficient processing.

The system supports automated file scanning and processing, allowing users to minimize manual effort while maintaining file organization. After processing, users can review scan results to verify file changes and identify any issues. Where inconsistencies such as incorrect naming or missing files are detected, the system enables users to resolve these issues through rule-based correction.

In addition, the system allows users to manage rules, including creating, modifying, removing, and configuring rules for automatic or manual processing. The system also provides reporting functionality, enabling users to analyse file processing outcomes and historical records. Finally, users can terminate their session securely through a logout function.

Functional Requirements

1. The system shall allow users to authenticate and connect to their OneDrive account.
2. The system shall allow users to select specific folders for analysis.
3. The system shall allow users to create, edit, and delete templates.
4. The system shall scan selected folders and compare them against a chosen template.
5. The system shall detect missing files based on the template.
6. The system shall detect incorrectly named files based on rules.
7. The system shall allow users to define rules for file renaming, moving, and copying.
8. The system shall automatically execute rule-based file operations.
9. The system shall display scan results on a dashboard.
10. The system shall generate a compliance report showing missing and corrected files.
11. The system shall store templates, rules, and logs in a local database.
12. The system shall allow users to re-run scans after applying fixes.

Use Cases

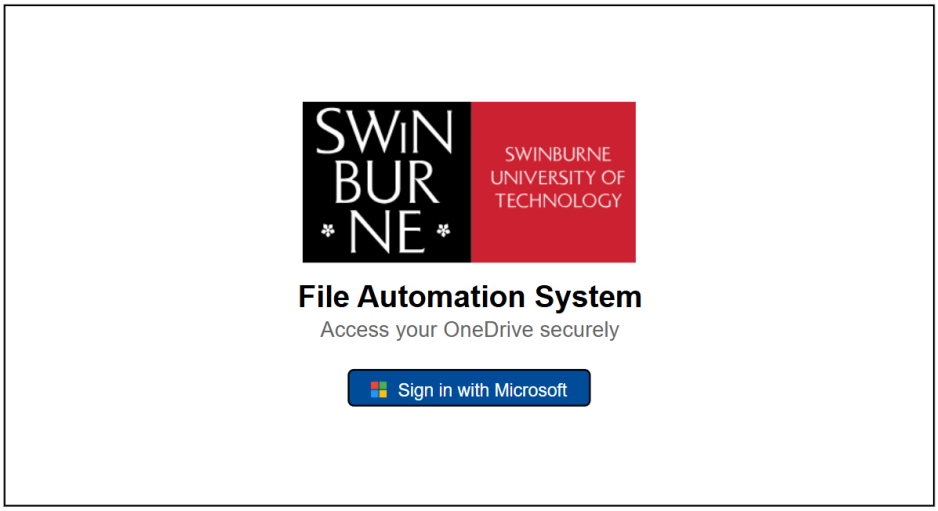


Figure 5: Login Page.

The login page allows users to sign in using their Microsoft account through API integration. When "Sign in with Microsoft" is selected, users are redirected to the Microsoft login interface to enter their credentials. If authentication is successful, the system logs the user in and redirects them to the dashboard. If authentication fails, an error message is displayed, and the user is prompted to try again. In cases such as network issues or API failure, the system notifies the user and allows another login attempt.

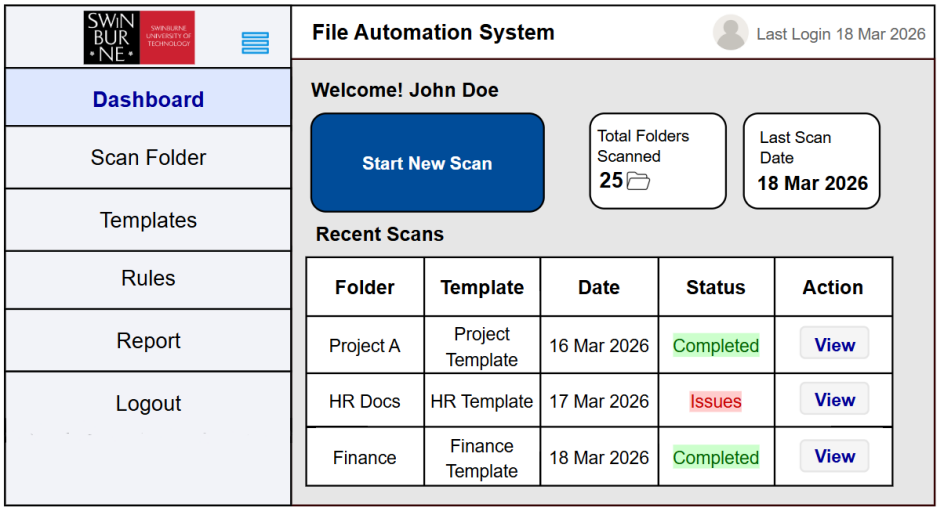


Figure 6: Dashboard View.

The dashboard serves as the main entry point of the system, providing access to key features. From here, users can start a new scan, view reports, or log out. Selecting "Start New Scan" navigates to the folder selection page, while "View Report" opens the reporting section. When "Logout" is selected, a confirmation message is displayed to prevent accidental logout. If confirmed, the system logs the user out and redirects them to the login page.

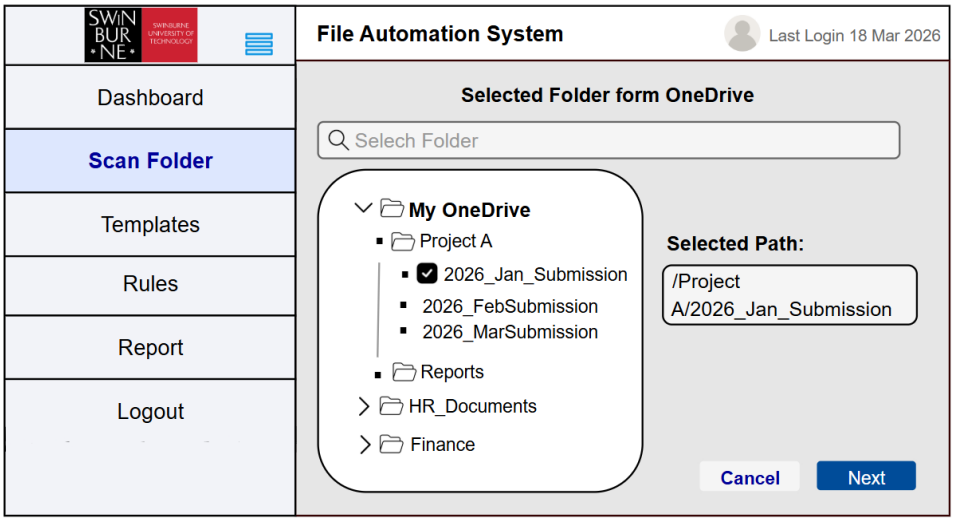


Figure 7: Folder Selection Page.

On the folder selection page, users choose the directory containing files to be processed. After selecting a folder, clicking "Next" proceeds to the template selection page. If the user decides not to continue, selecting "Cancel" returns to the dashboard. If no folder is selected, the system prompts the user to choose a valid folder before proceeding.

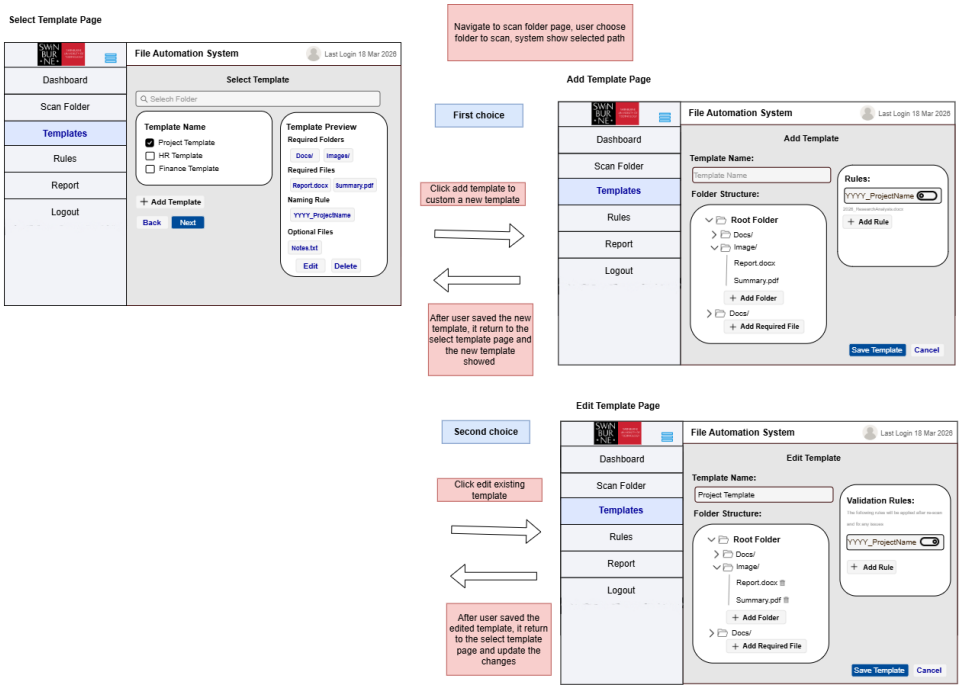


Figure 8: Template Selection Page.

The template selection page allows users to select and manage templates used for file processing. Users can choose an existing template and proceed by clicking “Next” or return to the dashboard by selecting “Back”. In addition, templates can be added, edited, or deleted directly from this page. Any updates made are immediately reflected in the template list to ensure the most current configuration is displayed.

When adding a new template, users are directed to the add template page, where they define the template name, folder structure, and associated rules. Upon clicking “Save”, the system validates the input and stores the new template before returning to the template selection page with the updated list. If the process is cancelled, no changes are saved. If required fields are missing or invalid, the system displays an error message and requires correction before submission.

Similarly, when editing an existing template, users can modify its configuration on the edit template page. After making changes, selecting “Save” updates the template and reflects the changes immediately. If cancelled, the original template remains unchanged. If an error occurs during saving, the system notifies the user and allows corrections before proceeding.

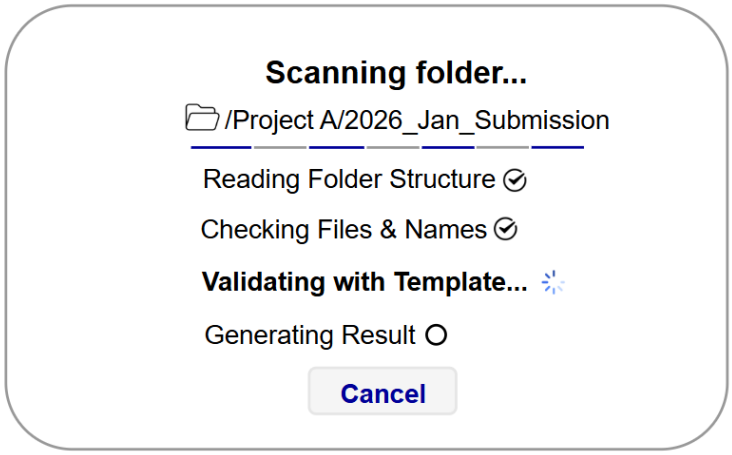


Figure 9: Scanning Pop Up.

During scanning, the system displays a pop up showing the selected folder path and the progress of file processing. Files are processed automatically based on the selected template. Status indicators such as "Not Started", "Scanning", and "Completed" provide real-time feedback. Once scanning is completed, the system navigates to the scan result page. If an error occurs, an appropriate message is displayed, and the user may retry the process.

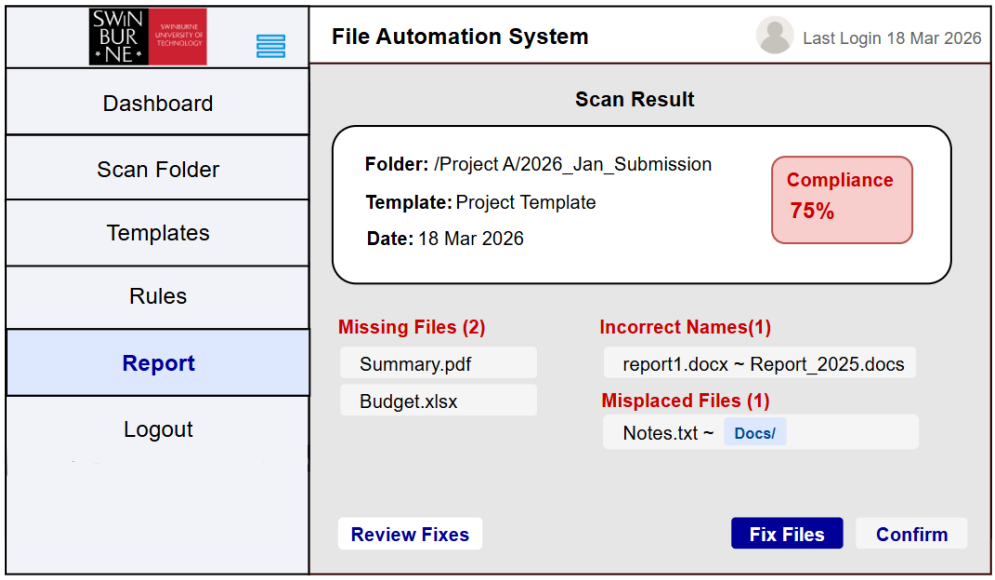


Figure 10: Scan Result Page.

The scan result page displays the outcomes of the processed files. Users can review the results, fix any issues, or confirm the final output. Selecting "Review" allows detailed inspection, while "Fix Files" navigates to the rules management page for adjustments. When "Confirm" is selected, the system finalizes the results, updates the records, and returns to the dashboard.

Figure 11: Rules Page.

The rules page enables users to manage how files are processed through configurable rules. Users can view all existing rules, including their priority, conditions, and status. Rules can be added, edited, deleted, or toggled on and off. When a rule is enabled, the system processes files automatically according to that rule; when disabled, manual intervention is required. Tooltips are provided to help users understand each rule's function. Any changes to rules trigger the system to reprocess files so that updated logic is applied.

When creating a new rule, users are directed to the add rule page to define rule details such as name, priority, conditions, and actions. After clicking "Save", the system validates the input, adds the rule, and triggers a rescan to apply the changes. If the action is cancelled, no changes are made. Invalid or incomplete inputs result in an error message requiring correction before saving.

Similarly, when editing an existing rule, users can update its configuration on the edit rule page. After confirming the changes, the system updates the rule and triggers a rescan. If cancelled, the original rule remains unchanged. If errors occur during saving, the system notifies the user and allows corrections.

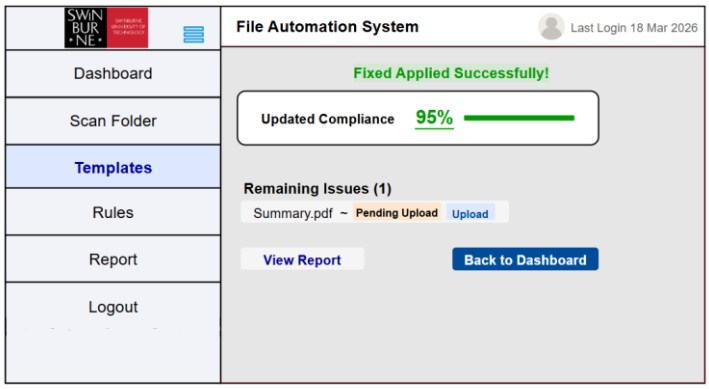


Figure 12: Final Result Page.

The final result page displays the updated outcomes after rules have been applied and files have been reprocessed. From this page, users can review the final results and either return to the dashboard or proceed to the report page for further analysis.

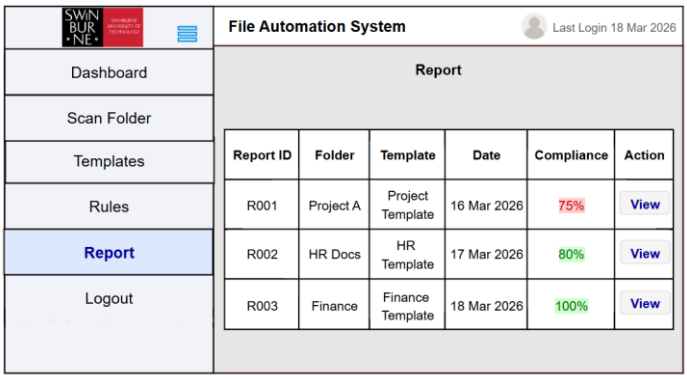


Figure 13: Report Page.

The report page provides a summary and detailed view of processed files. Users can filter reports based on templates or scan sessions and select a report to view more details. If no reports are available, the system displays an empty state message.

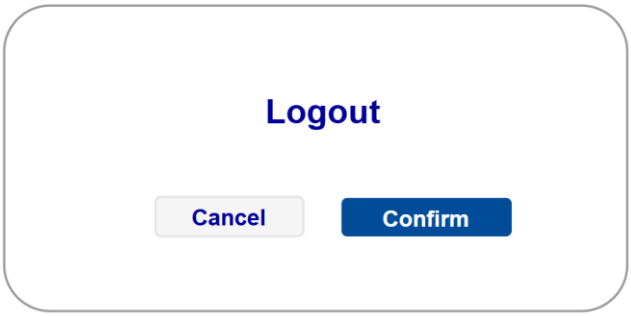


Figure 14: Log Out Pop Up.

When logging out, a confirmation message is displayed to ensure the action is intentional. If confirmed, the system logs the user out and redirects them to the login page. If cancelled, the user remains on the current page without any changes.

Summary of use cases

User Authentication

- The system authenticates users through Microsoft OAuth to ensure secure access. User credentials are validated via an external authentication service before access is granted. Upon successful authentication, a secure session is established, allowing the user to access system functionalities. If authentication fails, access is denied and the system provides feedback to allow the user to retry.

File Scanning Workflow

- The system supports an end-to-end file scanning process to organize and validate files. A valid folder must be provided as input, and appropriate templates must be available for processing. The system applies predefined template rules to analyze and process files within the selected folder. Upon completion, the system generates and stores processing results. If required inputs are missing or invalid, the system prevents execution and requests valid input.

Template Management

- The system allows templates to be created, modified, and removed to support flexible file recognition. Template data is validated before being applied to ensure correctness. Valid templates are stored and updated accordingly. If invalid data is detected, the system rejects the operation and provides appropriate feedback. If the operation is cancelled, no changes are applied.

Rule-Based File Correction

- The system enables file correction through rule-based processing. Rules define how files are handled and are validated before execution. Updated rules trigger reprocessing of affected files, and the system generates updated results reflecting the changes. If rules are invalid, execution is prevented. If rules are configured for manual processing, user intervention is required to complete the correction.

View Reports

- The system provides reporting functionality to support analysis of file processing activities. Stored processing records are retrieved and displayed, with filtering options available to refine results. Detailed report data is presented to support user evaluation. If no records are available, the system indicates that no data is present.

Logout

- The system allows users to securely terminate their session. Upon logout, the session is ended and access to system functionalities is restricted. If the logout action is not confirmed, the system maintains the current session without interruption.

Non-functional Requirements

Performance

- The system shall provide efficient performance by completing folder scanning within 10 seconds for folders containing up to 100 files. Additionally, the system shall respond to user interactions within 5 seconds to ensure smooth and responsive operation.

Usability

- The system shall offer a simple and intuitive user interface that is easy to navigate. Users should be able to complete key tasks, such as scanning a folder, within a minimal number of steps, ensuring accessibility for users with limited technical knowledge.

Security

- The system shall ensure secure access by implementing authentication through Microsoft Graph API using OAuth. User credentials shall not be stored locally, and all interactions with cloud storage shall be conducted through secure communication protocols.

Reliability

- The system shall maintain stable operation by handling errors gracefully, including API failures or invalid inputs. It shall provide clear error messages to users and log system activities to support debugging and issue resolution.

Maintainability

- The system shall be developed following Next.js coding standards and best practices. A modular code structure shall be adopted to facilitate future maintenance, updates, and scalability of the system.

Compatibility

- The system shall be compatible with modern web browsers such as Google Chrome and Microsoft Edge, ensuring consistent performance across commonly used platforms.

Constraints

Time

- The project must be completed within the academic timeline of Week 2 to Week 12. This limited timeframe restricts the scope of development and requires the team to prioritize core functionalities.

Technology Dependency

- The system must use Microsoft Graph API for accessing and managing OneDrive files. This creates a dependency on external API availability and reliability, which may affect system performance.

Compliance

- The system must comply with IT Application Team standards, which may restrict certain design choices, coding practices, and system architecture decisions.

Technical Expertise

- The development team has limited experience with Microsoft Graph API, which may introduce challenges in implementation and increase development complexity.

Network Dependency

- The system relies on stable internet connectivity to access cloud storage services. Any network disruption may affect system functionality and performance.

Deployment Environment

- The system is restricted to an internal or local environment and cannot be deployed as a public production system.

Data Storage

- The system uses lightweight storage solutions such as JSON or SQLite, which may limit scalability and performance for large-scale data handling.

Conclusion

In conclusion, this System Requirements Specification (SRS) has provided a comprehensive and structured overview of the proposed Template-Based Cloud File Organization and Compliance System. The document has clearly defined the problem of disorganized cloud storage environments and justified the need for an automated solution that enhances efficiency, consistency, and compliance in file management.

Through detailed sections covering system objectives, scope, architecture, planning, and requirements, the SRS establishes a solid foundation for system development. The adoption of modern technologies such as Next.js and Microsoft Graph API, combined with an Agile development methodology, ensures that the system is both technically feasible and aligned with current industry practices. Additionally, the inclusion of functional and non-functional requirements, risk management strategies, and clearly defined team roles supports a well-coordinated and manageable development process.

Overall, the proposed system is expected to significantly improve file organization by automating folder structuring, detecting inconsistencies, and applying rule-based corrections. Despite certain constraints and technical challenges, appropriate mitigation strategies have been identified to ensure successful project execution. This SRS therefore serves as a critical reference point for guiding the design, development, testing, and final delivery of the system, ensuring that it meets stakeholder expectations and project objectives.

References

Boogaard, K. (2024, September 12). How to write SMART goals (with examples). Work Life by Atlassian. https://www.atlassian.com/blog/productivity/how-to-write-smart-goals

GeeksforGeeks. (2025, July 23). Agile SDLC (Software Development Life Cycle). https://www.geeksforgeeks.org/software-engineering/agile-sdlc-software-development-life-cycle/

GeeksforGeeks. (2026, March 11). Functional and non functional requirements. GeeksforGeeks. https://www.geeksforgeeks.org/software-engineering/functional-vs-non-functional-requirements/

GeeksforGeeks. (2025, December 11). What is Risk Management?. https://www.geeksforgeeks.org/software-engineering/risk-management-software-engineering/

Kenton, W. (2025, December 19). SWOT: What Is It, How It Works, and How to Perform an Analysis. Investopedia. https://www.investopedia.com/terms/s/swot.asp

Klein, F., Pelzer, U., Schmuck, R. B., Malinka, T., Felsenstein, M., Denecke, T., Pratschke, J., & Bahra, M. (2018). Strengths, Weaknesses, Opportunities, and Threats of Centralized Pancreatic Surgery: a Single-Center Analysis of 3000 Consecutive Pancreatic Resections. *Journal of Gastrointestinal Surgery*, *23*(3), 492–502. https://doi.org/10.1007/s11605-018-3867-x

Martins, J. (2026, January 5). *What are SMART goals? Examples and templates*. Asana. https://asana.com/resources/smart-goals

Signed by:

Nilson Boon Lung
Fam Project Manager

Signed by:

Jason Thosmas Chew
Project Supervisor

Signed by:

Ye Zeng TAN
Team Leader

Signed by:

Angela
Team Member

Signed by:

Jia You LIM
Team Member

Signed by:

Qiung Hung CHONG
Team Member

Signed by:

Wen Xiun THEN
Team Member